More Dynamic Programming Examples

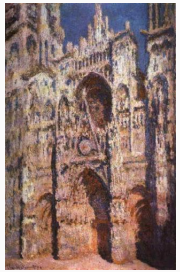
Prof. Timothy Sun

Art Thieves

Suppose there is an art gallery with ***n*** paintings in a row, and the ***i***th painting is worth p[***i***] dollars. What's the most value a burglar can steal, without stealing consecutive paintings?

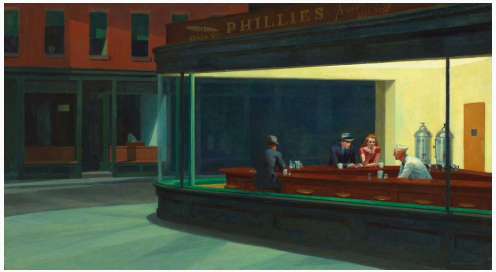
$8m

$12m



$11m

$1m



$2m

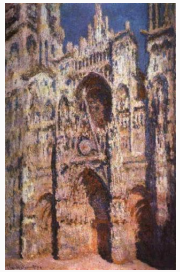
$9m

Art Thieves

Suppose there is an art gallery with ***n*** paintings in a row, and the ***i***th painting is worth p[***i***] dollars. What's the most value a burglar can steal, without stealing consecutive paintings?

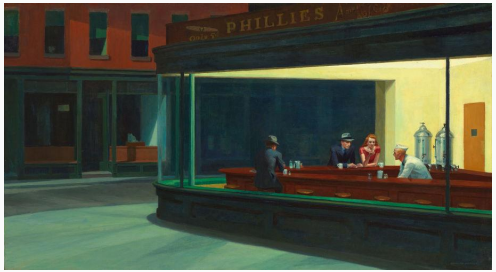
$8m

$12m



$11m

$1m



$2m

$9m

Art Thieves Recurrence

Either you steal or don’t steal the first painting.

Art Thieves Recurrence

Either you steal or don’t steal the first painting.

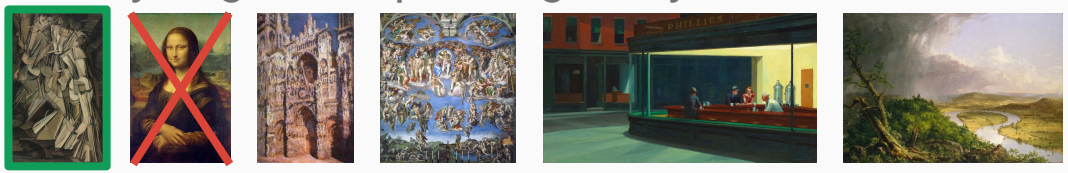
- If you don’t, you can still steal anything after it

Art Thieves Recurrence

Either you steal or don’t steal the first painting.

- If you don’t, you can still steal anything after it

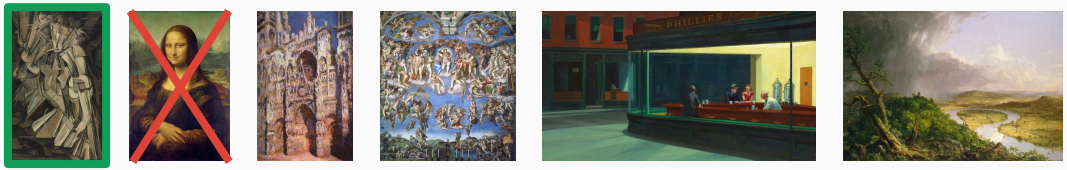
- If you do, you get that painting, but you can’t steal the second painting



Art Thieves Recurrence

M(***i***): most money for paintings ***i*** through ***n***

- M(***n***+1) = 0
- M(***i***) = max(M(***i***+1), p[***i***]+M(***i***+2))

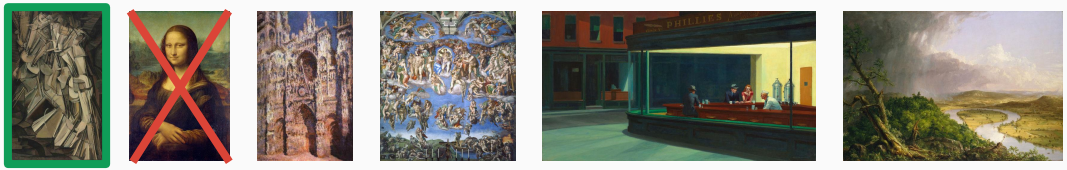


Art Thieves Recurrence

M(***i***): most money for paintings ***i*** through ***n***

- M(***n***+1) = M(***n***+2) = 0
- M(***i***) = max(M(***i***+1), p[***i***]+M(***i***+2))

Careful what you recurse on!



Art Thieves Backtracking

$\textsc{Steal}(p[1,\ldots,n], i)$:

    if $i = n+1$ or $n+2$

        return 0

    else

        return $\max(\textsc{Steal}(p, i+1), p[i] + \textsc{Steal}(p, i+2))$

Art Thieves Dynamic

Backtracking algorithm:

```
STEAL(p[1, ..., n], i):
    if i = n + 1 or n + 2
        return 0
    else
        return max(STEAL(p, i + 1), p[i] + STEAL(p, i + 2))
```

Storing subproblems:

Fill order:

Art Thieves Dynamic

Backtracking algorithm:

$$
\begin{aligned}
&\textsc{Steal}(p[1,\ldots,n], i):\\
&\qquad \textbf{if } i = n+1 \text{ or } n+2\\
&\qquad\qquad \textbf{return } 0\\
&\qquad \textbf{else}\\
&\qquad\qquad \textbf{return } \max(\textsc{Steal}(p, i+1), p[i] + \textsc{Steal}(p, i+2))
\end{aligned}
$$

Storing subproblems: M[1, …, ***n***+2]

Fill order: ***decreasing***

Art Thieves Dynamic

$\textsc{Steal}(p[1,\ldots,n])$:

initialize $M[1,\ldots,n+2]$

$M[n+1] := 0$

$M[n+2] := 0$

for $i = n,\ldots,1$

$M[i] := \max(M[i+1], p[i] + M[i+2])$

return $M[1]$

Art Thieves Dynamic

$\textsc{Steal}(p[1,\ldots,n])$:

initialize $M[1,\ldots,n+2]$

$M[n+1] := 0$

$M[n+2] := 0$

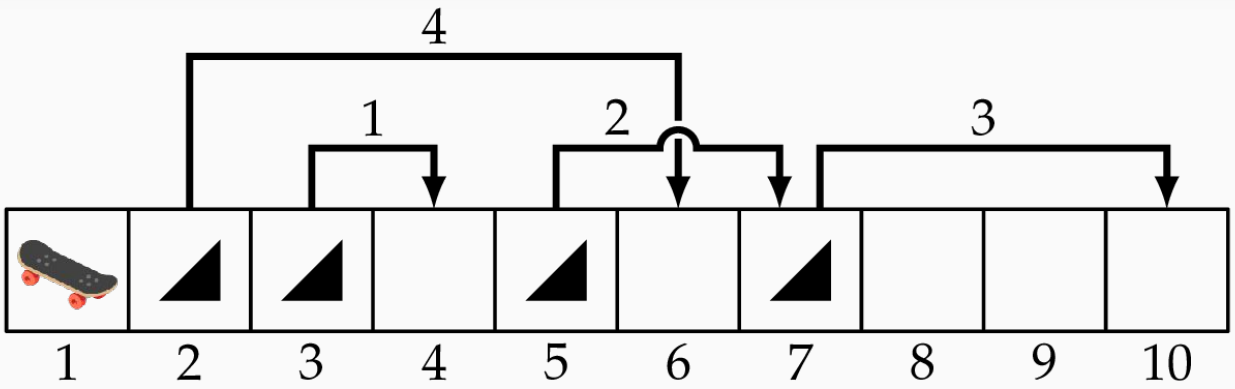
for $i = n,\ldots,1$

$M[i] := \max(M[i+1], p[i] + M[i+2])$

return $M[1]$

Runtime: $\Theta(\boldsymbol{n})$

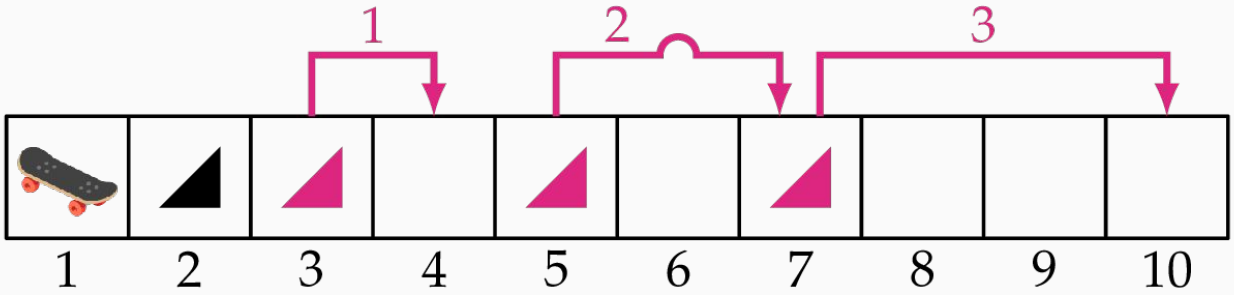
Skateboarding



Suppose jumping off each ramp keeps you in the air some distance.

What's the ***most number of jumps*** you can do?

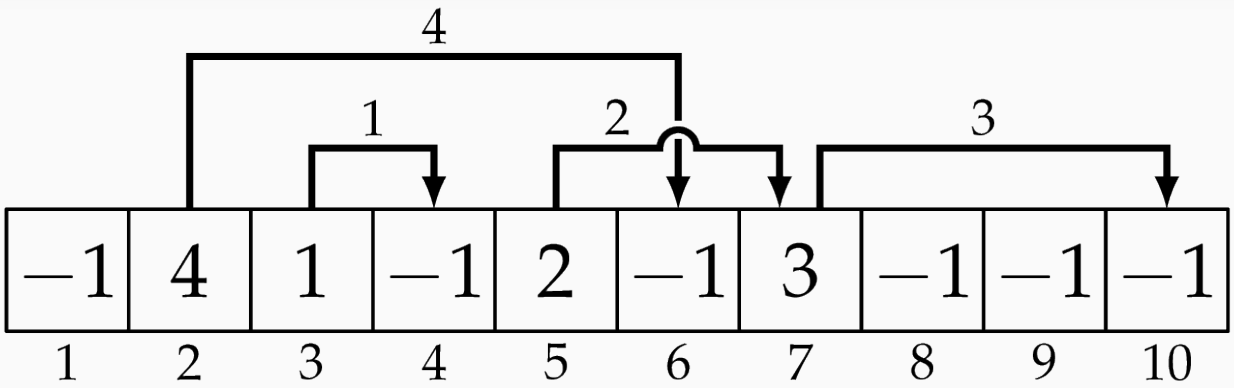
Skateboarding



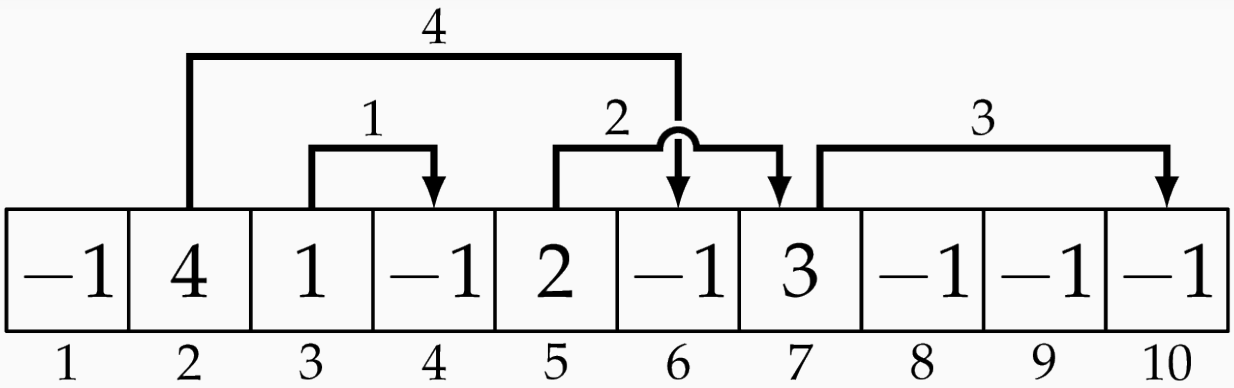
Suppose jumping off each ramp keeps you in the air some distance.

What's the ***most number of jumps*** you can do? ***3***

Skateboarding



Skateboarding Recurrence



$$J(i) = \begin{cases} 0 & \text{if } i = n \\ J(i+1) & \text{if } R[i] = -1 \\ \max(J(i+1), 1 + J(i + R[i])) & \text{otherwise} \end{cases}$$

Skateboarding Dynamic

$\textsc{Jump}(R[1,\ldots,n])$:

initialize $J[1,\ldots,n]$

$J[n] := 0$

for $i = n-1,\ldots,1$

  if $R[i] = -1$

    $J[i] := J[i+1]$

  else

    $J[i] := \max(J[i+1], 1 + J(i + R[i]))$

return $J[1]$

Runtime: $\Theta(\boldsymbol{n})$

Subsequences

Unlike subarrays, ***subsequences*** don't need to be contiguous:

SALE**SM**A**N**

The string "SSMN" is a subsequence of "SALESMAN"

Subsequences

Unlike subarrays, ***subsequences*** don't need to be contiguous:

SALE**SM**A**N**

The string "SSMN" is a subsequence of "SALESMAN"

How many ways to select a subsequence from a string with ***n*** characters?

Subsequences

Unlike subarrays, ***subsequences*** don't need to be contiguous:

SALE**SM**A**N**

The string "SSMN" is a subsequence of "SALESMAN"

How many ways to select a subsequence from a string with ***n*** characters?

2^n (think subsets)

Longest Common Subsequence

Given two strings, what is their **longest** common subsequence?

A**BA**C**US**

BAUHAU**S**

Longest Common Subsequence

A few more examples of strings and their LCSs:

PARTY APTLY or PARTY APTLY

ALGORITHM ALTRUISTIC

CGATGATC TGGCATAC

Longest Common Subsequence

Step 1: Design a backtracking algorithm (for size of LCS)

Longest Common Subsequence

Step 1: Design a backtracking algorithm (for size of LCS)

- Subproblems: substrings

Longest Common Subsequence

Step 1: Design a backtracking algorithm (for size of LCS)

- Subproblems: substrings

What's the "nibble"?

Longest Common Subsequence

Step 1: Design a backtracking algorithm (for size of LCS)

- Subproblems: substrings

What's the "nibble"?

- Let's see what we can do with the ***last*** characters of each string.

Longest Common Subsequence

Let's see what we can do with the last characters of each string.

PRIEST**S** SPIRE**S**

Last chars are the same. Do they belong to ***every*** LCS?

Longest Common Subsequence

Let's see what we can do with the last characters of each string.

PRIEST**S** SPIRE**S**

Last chars are the same. Do they belong to ***every*** LCS? ***No***:

PRI**ES**TS S**P**I**RES**

Longest Common Subsequence

Let's see what we can do with the last characters of each string.

PRIEST**S** SPIRE**S**

Last chars are the same. Do they belong to ***every*** LCS? ***No***:

PRIESTS **SPIRES**

...but we can always modify an LCS to get one where both last chars belong.

Longest Common Subsequence

Let’s see what we can do with the last characters of each string.

PRIES**T** SPIR**E**

Last chars are different. Do either belong to an LCS?

Longest Common Subsequence

Let’s see what we can do with the last characters of each string.

PRIES**T** SPIR**E**

Last chars are different. Do either belong to an LCS? **Maybe, but not both.**

Longest Common Subsequence

Let's see what we can do with the last characters of each string.

`PRIEST` `SPIRE`

Last chars are different. Do either belong to an LCS? **Maybe, but not both.**

Recurse on (`PRIES`, `SPIRE`) and (`PRIEST`, `SPIR`) and see which is bigger.

LCS Recurrence

Given two strings $\boldsymbol{S}[1, \ldots, \boldsymbol{m}]$, $\boldsymbol{T}[1, \ldots, \boldsymbol{n}]$

$L(\boldsymbol{i}, \boldsymbol{j})$ = length of LCS of $\boldsymbol{S}[1, \ldots, \boldsymbol{i}]$ and $\boldsymbol{T}[1, \ldots, \boldsymbol{j}]$

LCS Recurrence

Given two strings $\boldsymbol{S}[1, \ldots, \boldsymbol{m}]$, $\boldsymbol{T}[1, \ldots, \boldsymbol{n}]$

$L(\boldsymbol{i}, \boldsymbol{j})$ = length of LCS of $\boldsymbol{S}[1, \ldots, \boldsymbol{i}]$ and $\boldsymbol{T}[1, \ldots, \boldsymbol{j}]$

To find: ***base cases***, ***recurrence***

LCS Recurrence

Base cases: ***S*** = empty string, ***T*** = anything (or vice versa)

-

LCS Recurrence

Base cases: ***S*** = empty string, ***T*** = anything (or vice versa)

- $L(0, j) = L(i, 0) = 0$

LCS Recurrence

If last characters match, remove both and recurse on the rest

- $L(\boldsymbol{i}, \boldsymbol{j}) =$

If last characters don’t match, take the better of two one-char deletions

- $L(\boldsymbol{i}, \boldsymbol{j}) =$

LCS Recurrence

If last characters match, remove both and recurse on the rest

- $L(i, j) = 1 + L(i-1, j-1)$

If last characters don't match, take the better of two one-char deletions

- $L(i, j) = \max(L(i-1, j), L(i, j-1))$

LCS Recurrence

The final recurrence:

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

LCS Backtracking

$\text{LCS}(S[1,\ldots,m], T[1,\ldots,n], i, j)$:

    if $i = 0$ or $j = 0$

        return 0

    else if $S[i] = T[j]$

        return $1 + \text{LCS}(S, T, i-1, j-1)$

    else

        return $\max(\text{LCS}(S, T, i-1, j), \text{LCS}(S, T, i, j-1))$

LCS Dynamic Programming

Step 1: Design a backtracking alg.

```
LCS(S[1,...,m], T[1,...,n], i, j):
    if i = 0 or j = 0
        return 0
    else if S[i] = T[j]
        return 1 + LCS(S, T, i − 1, j − 1)
    else
        return max(LCS(S, T, i − 1, j), LCS(S, T, i, j − 1))
```

Step 2: Store the subproblems

LCS Dynamic Programming

Step 1: Design a backtracking alg.

```
LCS(S[1,...,m], T[1,...,n], i, j):
    if i = 0 or j = 0
        return 0
    else if S[i] = T[j]
        return 1 + LCS(S, T, i − 1, j − 1)
    else
        return max(LCS(S, T, i − 1, j), LCS(S, T, i, j − 1))
```

Step 2: Store the subproblems - 2D array ***L***[0, …, ***m***][0, …, ***n***]

LCS Dynamic Programming

Step 1: Design a backtracking alg.

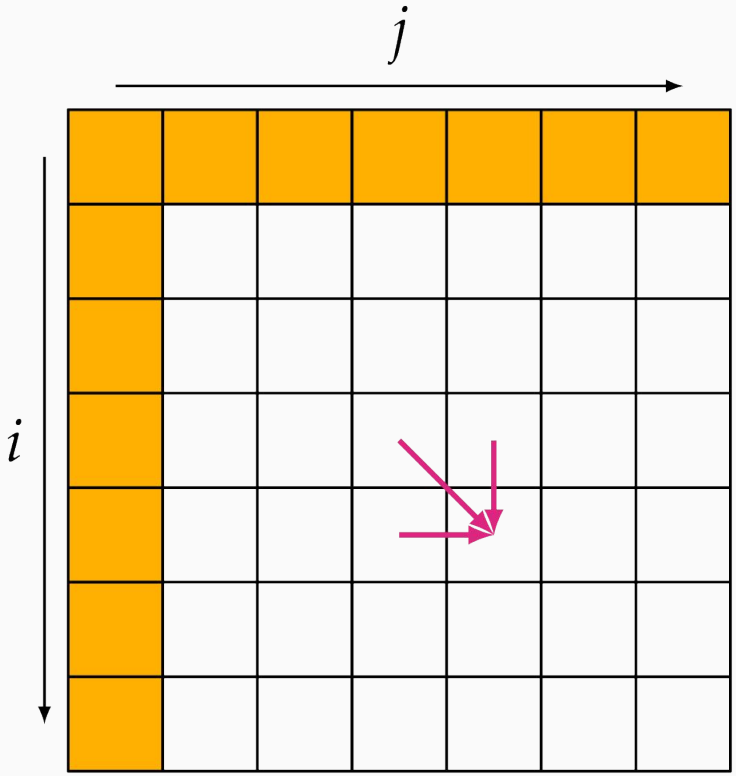
```
LCS(S[1,...,m], T[1,...,n], i, j):
    if i = 0 or j = 0
        return 0
    else if S[i] = T[j]
        return 1 + LCS(S, T, i − 1, j − 1)
    else
        return max(LCS(S, T, i − 1, j), LCS(S, T, i, j − 1))
```

Step 2: Store the subproblems - 2D array ***L***[0, ..., ***m***][0, ..., ***n***]

Step 3: Fill order - ???

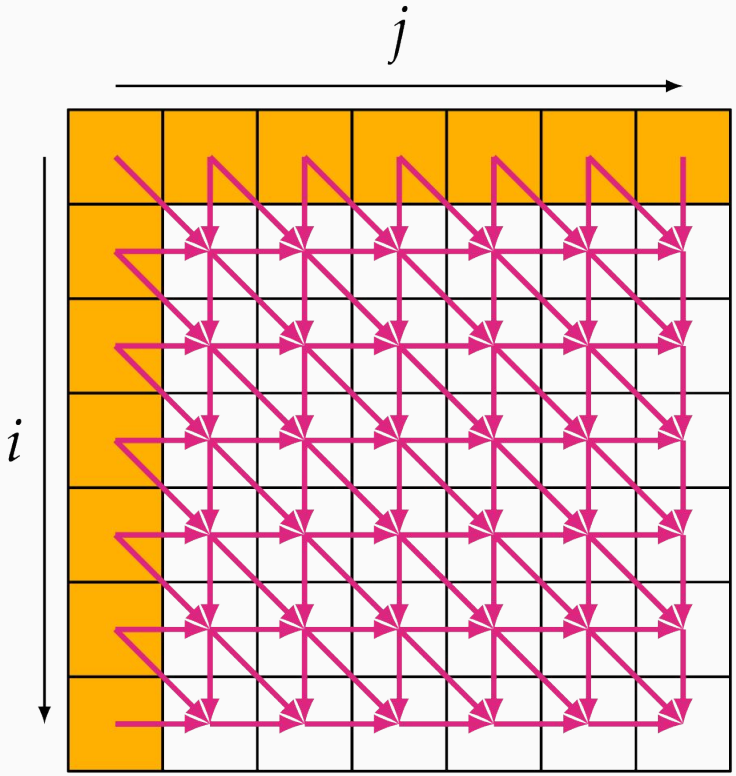
LCS - Fill order

		B	A	U	H	A	U	S
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	1
A	0	1	2	2	2	2	2	2
C	0	1	2	2	2	2	2	2
U	0	1	2	3	3	3	3	3
S	0	1	2	3	3	3	3	4



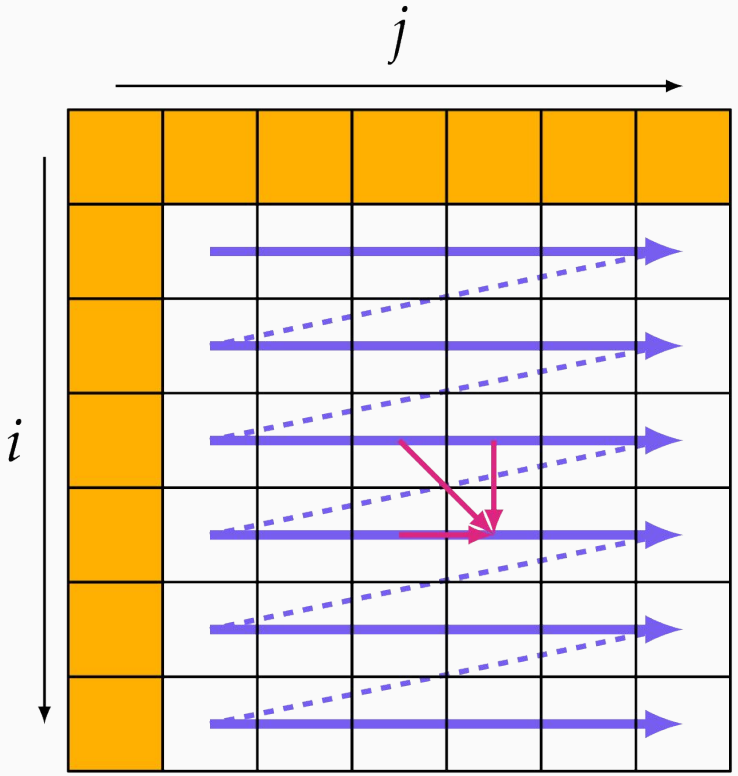
LCS - Fill order

		B	A	U	H	A	U	S
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	1
A	0	1	2	2	2	2	2	2
C	0	1	2	2	2	2	2	2
U	0	1	2	3	3	3	3	3
S	0	1	2	3	3	3	3	4



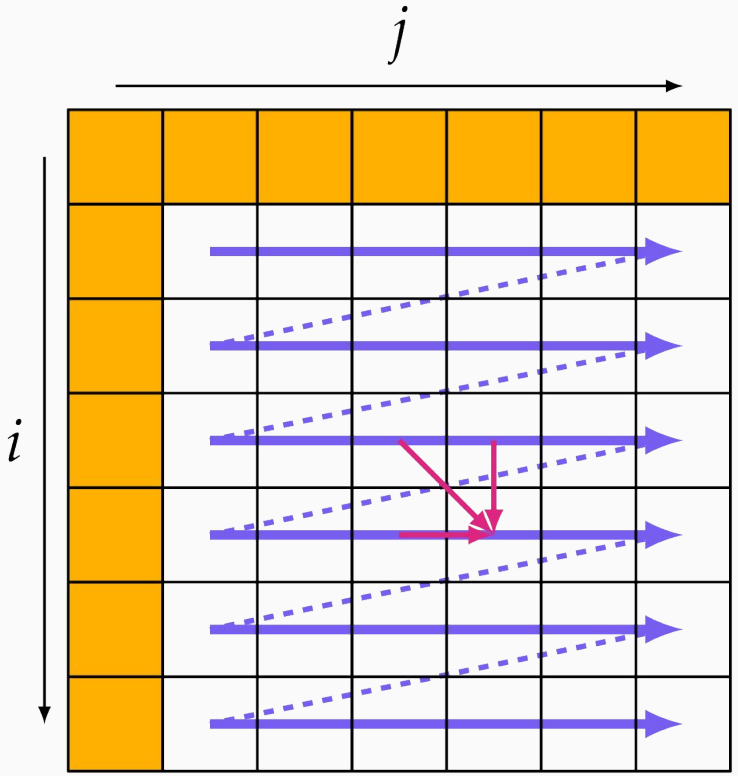
LCS - Fill order

		B	A	U	H	A	U	S
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	1
A	0	1	2	2	2	2	2	2
C	0	1	2	2	2	2	2	2
U	0	1	2	3	3	3	3	3
S	0	1	2	3	3	3	3	4



LCS - Fill order

		B	A	U	H	A	U	S
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	1
A	0	1	2	2	2	2	2	2
C	0	1	2	2	2	2	2	2
U	0	1	2	3	3	3	3	3
S	0	1	2	3	3	3	3	4



"like reading a book [in English]"

LCS Dynamic Programming

```
LCS(S[1,...,m], T[1,...,n]):
    initialize 2D array L[0,...,m][0,...,n]
    for i = 0,...,m
        L[i][0] := 0
    for j = 0,...,n
        L[0][j] := 0
    for i = 1,...,m
        for j = 1,...,n
            if S[i] = T[j]
                L[i][j] := 1 + L[i-1][j-1]
            else
                L[i][j] := max(L[i-1][j], L[i][j-1])
    return L[m][n]
```

LCS Dynamic Programming

```
LCS(S[1,...,m], T[1,...,n]):
    initialize 2D array L[0,...,m][0,...,n]
    for i = 0,...,m
        L[i][0] := 0
    for j = 0,...,n
        L[0][j] := 0
    for i = 1,...,m
        for j = 1,...,n
            if S[i] = T[j]
                L[i][j] := 1 + L[i-1][j-1]
            else
                L[i][j] := max(L[i-1][j], L[i][j-1])
    return L[m][n]
```

Runtime: $\Theta(\boldsymbol{mn})$

LCS Reconstruction

		R	E	S	T	R	A	I	N
	0	0	0	0	0	0	0	0	0
T	0	0	0	0	1	1	1	1	1
A	0	0	0	0	1	1	2	2	2
W	0	0	0	0	1	1	2	2	2
N	0	0	0	0	1	1	2	2	3
I	0	0	0	0	1	1	2	3	3
E	0	0	1	1	1	1	2	3	3
R	0	1	1	1	1	2	2	3	3

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

- If there's a match, step diagonally

LCS Reconstruction

		R	E	S	T	R	A	I	N
	0	0	0	0	0	0	0	0	0
T	0	0	0	0	1	1	1	1	1
A	0	0	0	0	1	1	2	2	2
W	0	0	0	0	1	1	2	2	2
N	0	0	0	0	1	1	2	2	3
I	0	0	0	0	1	1	2	3	3
E	0	0	1	1	1	1	2	3	3
R	0	1	1	1	1	2	2	3	3

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

- If there's a match, step diagonally
- Otherwise, move up, then left (in same layer) until you find another

LCS Reconstruction

		R	E	S	T	R	A	I	N
	0	0	0	0	0	0	0	0	0
T	0	0	0	0	1	1	1	1	1
A	0	0	0	0	1	1	2	2	2
W	0	0	0	0	1	1	2	2	2
N	0	0	0	0	1	1	2	2	3
I	0	0	0	0	1	1	2	3	3
E	0	0	1	1	1	1	2	3	3
R	0	1	1	1	1	2	2	3	3

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

- If there's a match, step diagonally
- Otherwise, move up, then left (in same layer) until you find another

LCS Reconstruction

		R	E	S	T	R	A	I	N
	0	0	0	0	0	0	0	0	0
T	0	0	0	0	1	1	1	1	1
A	0	0	0	0	1	1	2	2	2
W	0	0	0	0	1	1	2	2	2
N	0	0	0	0	1	1	2	2	3
I	0	0	0	0	1	1	2	3	3
E	0	0	1	1	1	1	2	3	3
R	0	1	1	1	1	2	2	3	3

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

- If there's a match, step diagonally
- Otherwise, move up, then left (in same layer) until you find another

LCS Reconstruction

		R	E	S	T	R	A	I	N
	0	0	0	0	0	0	0	0	0
T	0	0	0	0	1	1	1	1	1
A	0	0	0	0	1	1	2	2	2
W	0	0	0	0	1	1	2	2	2
N	0	0	0	0	1	1	2	2	3
I	0	0	0	0	1	1	2	3	3
E	0	0	1	1	1	1	2	3	3
R	0	1	1	1	1	2	2	3	3

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

- If there's a match, step diagonally
- Otherwise, move up, then left (in same layer) until you find another

LCS Reconstruction

		R	E	S	T	R	A	I	N
	0	0	0	0	0	0	0	0	0
T	0	0	0	0	1	1	1	1	1
A	0	0	0	0	1	1	2	2	2
W	0	0	0	0	1	1	2	2	2
N	0	0	0	0	1	1	2	2	3
I	0	0	0	0	1	1	2	3	3
E	0	0	1	1	1	1	2	3	3
R	0	1	1	1	1	2	2	3	3

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

- If there's a match, step diagonally
- Otherwise, move up, then left (in same layer) until you find another

LCS Reconstruction

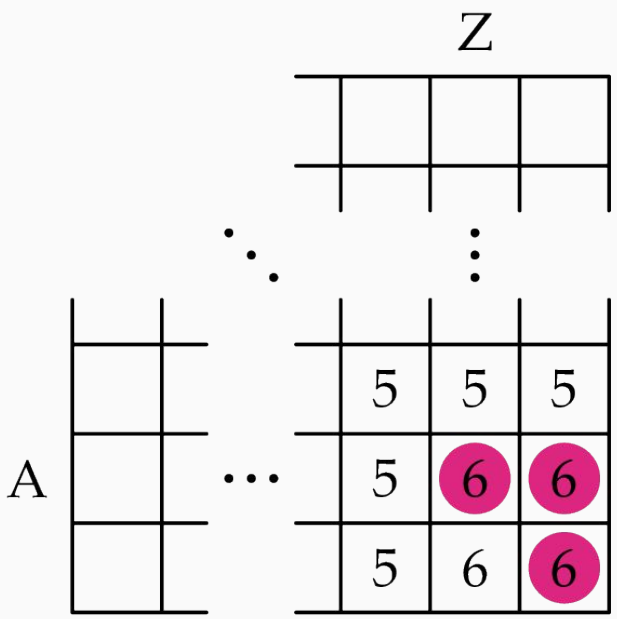
		R	E	S	T	R	A	I	N
	0	0	0	0	0	0	0	0	0
T	0	0	0	0	1	1	1	1	1
A	0	0	0	0	1	1	2	2	2
W	0	0	0	0	1	1	2	2	2
N	0	0	0	0	1	1	2	2	3
I	0	0	0	0	1	1	2	3	3
E	0	0	1	1	1	1	2	3	3
R	0	1	1	1	1	2	2	3	3

$$L(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + L(i-1, j-1) & \text{if } S[i] = T[j] \\ \max(L(i, j-1), L(i-1, j)) & \text{otherwise} \end{cases}$$

- If there's a match, step diagonally
- Otherwise, move up, then left (in same layer) until you find another

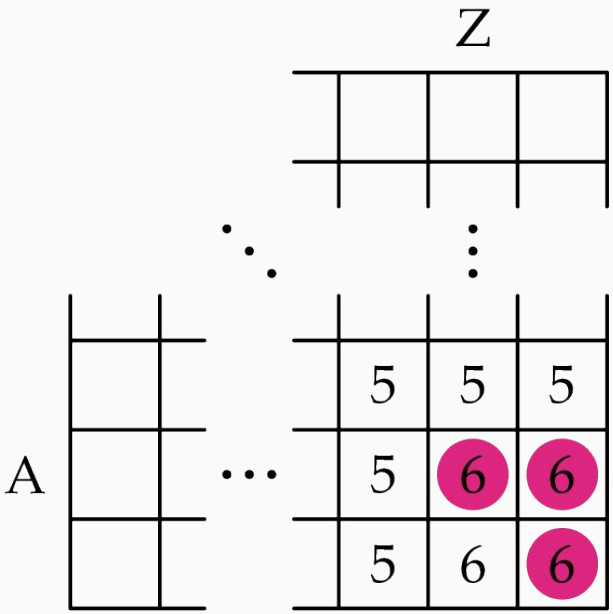
(one possible) LCS: **TAN**

LCS Reconstruction



Does this process ever get stuck?

LCS Reconstruction



Does this process ever get stuck?

This situation is ***impossible***.

LCS Reconstruction

```
LCSReconstruct(S[1,...,m], T[1,...,n]):
    // fill in the table L[0,...,m][0,...,n]
    i := m, j := n
    while i > 0 and j > 0
        if S[i] = T[j]
            print S[i], i, j
            i := i − 1, j := j − 1
        else
            if L[i][j] = L[i − 1][j]: i := i − 1
            else: j := j − 1
```

Edit Distance

How quickly can you change “ALERT” into “LATER”, with the following operations?

- Inserting a character
- Deleting a character
- Substituting a character

Edit Distance

How quickly can you change “ALERT” into “LATER”, with the following operations?

- Inserting a character
- Deleting a character
- Substituting a character

3 operations:

- ALER**T** → ALER (deletion) → **L**ALER (insertion) → LA**T**ER (substitution)

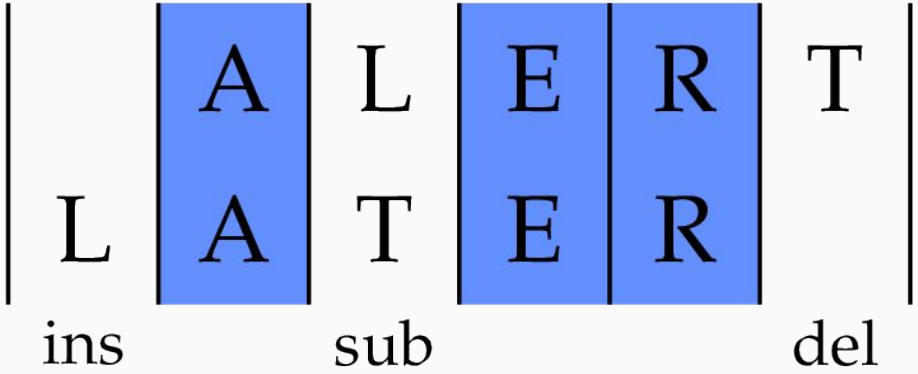
Edit Distance

Both LCS and edit distance can be thought of as “alignment problems”:

	A	L	E	R	T
L	A	T	E	R	
ins		sub			del

Edit Distance

Both LCS and edit distance can be thought of as “alignment problems”:



Edit distance has a similar solution (what happens for those last characters?)

ED Recurrence

Base cases: ***S*** = empty string, ***T*** = anything (or vice versa)

-

ED Recurrence

Base cases: ***S*** = empty string, ***T*** = anything (or vice versa)

- $E(0, \boldsymbol{j}) = \boldsymbol{j}$, $E(\boldsymbol{i}, 0) = \boldsymbol{i}$

ED Recurrence

When both strings are non-empty, take the best operation:

- $E(\boldsymbol{i}, \boldsymbol{j})$ = min(insertion, deletion, substitution)

ED Recurrence

When both strings are non-empty, take the best operation:

- $E(\boldsymbol{i}, \boldsymbol{j})$ = min(insertion, deletion, substitution)
  - Insertion = $E(\boldsymbol{i}, \boldsymbol{j}\text{-}1) + 1$
  - Deletion = $E(\boldsymbol{i}\text{-}1, \boldsymbol{j}) + 1$

ED Recurrence

When both strings are non-empty, take the best operation:

- $E(\boldsymbol{i}, \boldsymbol{j})$ = min(insertion, deletion, substitution)
  - Insertion = $E(\boldsymbol{i}, \boldsymbol{j}\text{-}1) + 1$
  - Deletion = $E(\boldsymbol{i}\text{-}1, \boldsymbol{j}) + 1$
  - Substitution = $E(\boldsymbol{i}\text{-}1, \boldsymbol{j}\text{-}1) + 1$

ED Recurrence

When both strings are non-empty, take the best operation:

- $E(i, j)$ = min(insertion, deletion, substitution)
  - Insertion = $E(i, j\text{-}1) + 1$
  - Deletion = $E(i\text{-}1, j) + 1$
  - Substitution =
    - $E(i\text{-}1, j\text{-}1) + 1$ if $S[i] \neq T[j]$
    - $E(i\text{-}1, j\text{-}1)$ if $S[i] = T[j]$ (if the characters match, no cost)

ED Recurrence

The final recurrence:

$$E(i,j) = \begin{cases} i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ \min(E(i,j-1)+1, & \\ \qquad E(i-1,j)+1, & \\ \qquad E(i-1,j-1)+[S[i] \neq T[j]] & \text{otherwise} \end{cases}$$

ED Backtracking

$\textsc{Edit}(S[1,\ldots,m], T[1,\ldots,n], i, j)$:

```
    if i = 0
        return j
    else if j = 0
        return i
    else
        return min(EDIT(S, T, i − 1, j) + 1,
                   EDIT(S, T, i, j − 1) + 1,
                   EDIT(S, T, i − 1, j − 1) + [S[i] ≠ T[j]])
```

ED Dynamic Programming

The “shape” of base cases and dependencies is exactly the same as LCS

ED Dynamic Programming

The “shape” of base cases and dependencies is exactly the same as LCS

- Let’s reuse the same fill order

ED Dynamic Programming

$\textsc{Edit}(S[1,\ldots,m], T[1,\ldots,n])$:

initialize 2D array $E[0,\ldots,m][0,\ldots,n]$

for $i = 0,\ldots,m$

$E[i][0] := i$

for $j = 0,\ldots,n$

$E[0][j] := j$

for $i = 1,\ldots,m$

for $j = 1,\ldots,n$

$$E[i][j] := \min(E[i-1][j]+1,\; E[i][j-1]+1,\; E[i-1][j-1]+[S[i] \neq T[j]])$$

return $E[m][n]$

ED Dynamic Programming

$\textsc{Edit}(S[1,\ldots,m], T[1,\ldots,n])$:

initialize 2D array $E[0,\ldots,m][0,\ldots,n]$

for $i = 0,\ldots,m$

$E[i][0] := i$

for $j = 0,\ldots,n$

$E[0][j] := j$

for $i = 1,\ldots,m$

for $j = 1,\ldots,n$

$$E[i][j] := \min(E[i-1][j]+1,\; E[i][j-1]+1,\; E[i-1][j-1]+[S[i] \neq T[j]])$$

return $E[m][n]$

Runtime: $\Theta(\boldsymbol{mn})$

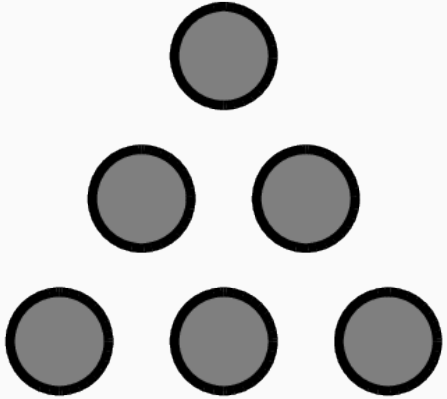
Recursive definition for winning

If the game has already been won/lost, nothing to calculate.

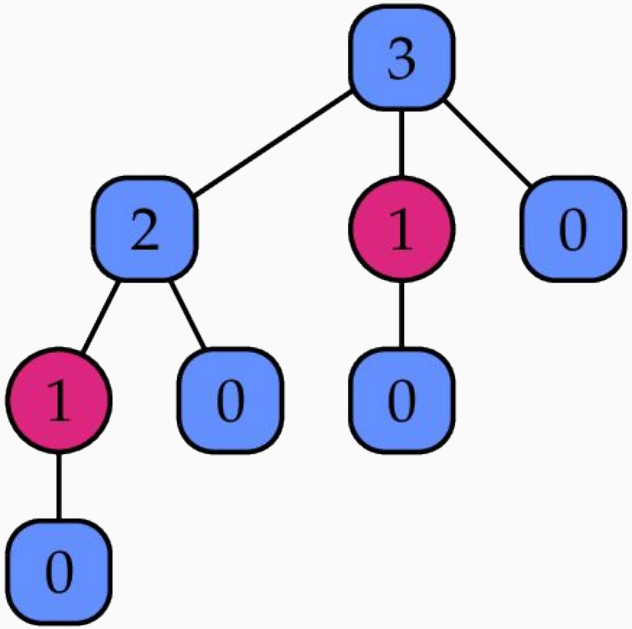
A state is ***winning*** if there is a move the player can make that makes it ***losing*** for the other player.

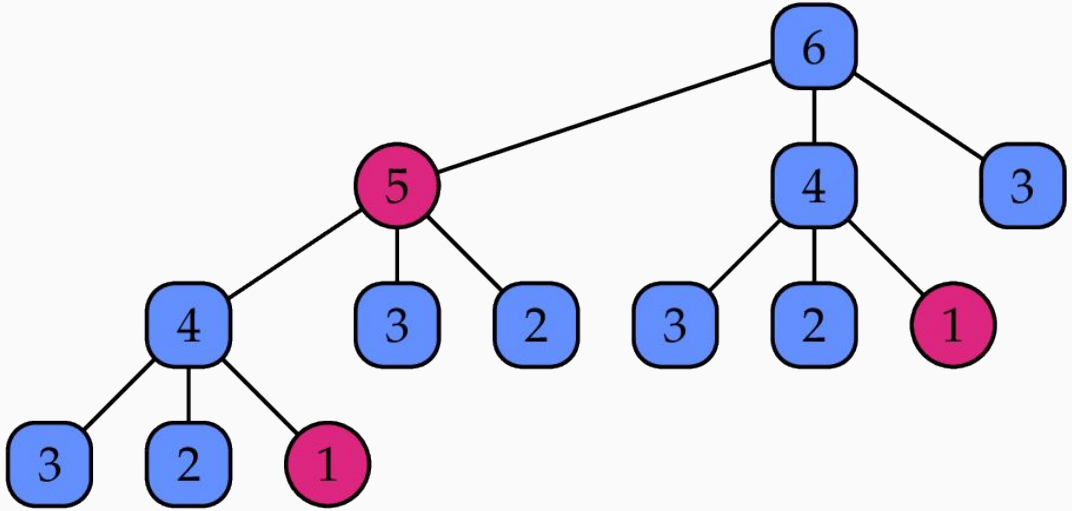
Nim - variant 1

First version: 1 pile, 1-3 stones per turn, person who takes last loses



Nim 1 - game trees





Nim 1 - DP solver

$\textsc{Nim1}(n)$:

initialize $W[0, \ldots, n]$

$W[0] = \textsc{True}$

$W[1] = \textsc{False}$

$W[2] = \textsc{True}$

for $i = 3, \ldots, n$

$W[i] :=$ not $(W[i-1]$ and $W[i-2]$ and $W[i-3])$

return $W[n]$

Nim 1

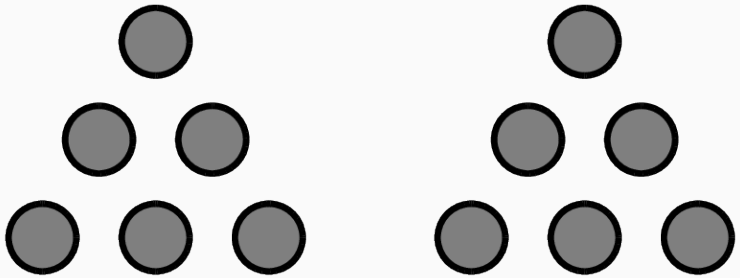
Actually there's a really simple strategy:

$$\textsc{Nim1}(n):$$
$$\quad \textbf{return } n \bmod 4 \neq 1$$

Nim - variant 2

Second version:

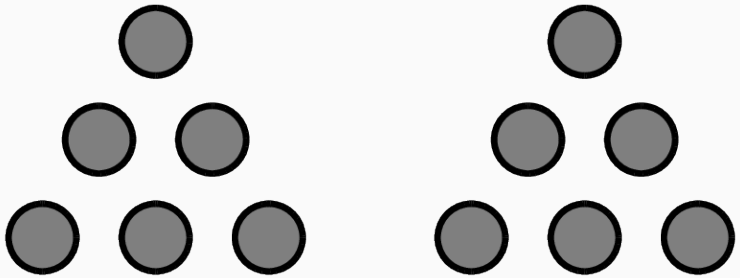
- 2 piles
- any (positive) # of stones from 1 pile
- player who takes last stone wins



Nim - variant 2

Second version:

- 2 piles
- any (positive) # of stones from 1 pile
- player who takes last stone wins



Many possible moves now, so we need to loop through the possibilities

Backtracking to Dynamic

```
NIM2(m, n):
    if m = 0 and n = 0
        return FALSE
    for i = 1, ..., m
        if not NIM2(m − i, n)
            return TRUE
    for j = 1, ..., n
        if not NIM2(m, n − j)
            return TRUE
    return FALSE
```

Backtracking to Dynamic

```
NIM2(m, n):
    if m = 0 and n = 0
        return FALSE
    for i = 1, ..., m
        if not NIM2(m − i, n)
            return TRUE
    for j = 1, ..., n
        if not NIM2(m, n − j)
            return TRUE
    return FALSE
```

```
NIM2(m, n):
    initialize W[0, ..., m][0, ..., n]
    for a = 0, ..., m
        for b = 0, ..., n
            if a = 0 and b = 0
                W[a][b] := FALSE
            else
                W[a][b] := FALSE
                for i = 1, ..., a
                    if not W[a − i][b]
                        W[a][b] := TRUE
                for j = 1, ..., b
                    if not W[a][b − j]
                        W[a][b] := TRUE
```

A mirroring strategy

Once again, an easy strategy:

$$\textsc{Nim2}(m, n):$$
$$\quad \textbf{return } m \neq n$$

Coin optimization

For any amount of money, what's the fewest number of coins needed? (25, 10, 5, 1)

Coin optimization

For any amount of money, what's the fewest number of coins needed? (25, 10, 5, 1)

$$MC(0) = 0$$

$$\begin{aligned} MC(i) = \min(&1 + MC(i-1), \\ &1 + MC(i-5), \\ &1 + MC(i-10), \\ &1 + MC(i-25)) \end{aligned}$$

Coin optimization - backtracking

```
COIN(i):
    if i = 0
        return 0
    else
        best := 1+COIN(i−1)
        if i ≥ 5
            best := min(best, 1+COIN(i−5))
        if i ≥ 10
            best := min(best, 1+COIN(i−10))
        if i ≥ 25
            best := min(best, 1+COIN(i−25))
        return best
```

Coin optimization - dynamic

```
COIN(i):
    if i = 0
        return 0
    else
        best := 1+COIN(i−1)
        if i ≥ 5
            best := min(best, 1+COIN(i−5))
        if i ≥ 10
            best := min(best, 1+COIN(i−10))
        if i ≥ 25
            best := min(best, 1+COIN(i−25))
        return best
```

```
COIN(n):
    initialize MC[0, ..., n]
    MC[0] := 0
    for i = 1, ..., n
        best := 1+COIN(i−1)
        if i ≥ 5
            best := min(best, 1+COIN(i−5))
        if i ≥ 10
            best := min(best, 1+COIN(i−10))
        if i ≥ 25
            best := min(best, 1+COIN(i−25))
        MC[i] := best
    return MC[n]
```

Coin optimization - dynamic

```
COIN(i):
    if i = 0
        return 0
    else
        best := 1+COIN(i−1)
        if i ≥ 5
            best := min(best, 1+COIN(i−5))
        if i ≥ 10
            best := min(best, 1+COIN(i−10))
        if i ≥ 25
            best := min(best, 1+COIN(i−25))
        return best
```

```
COIN(n):
    initialize MC[0, ..., n]
    MC[0] := 0
    for i = 1, ..., n
        best := 1+COIN(i−1)
        if i ≥ 5
            best := min(best, 1+COIN(i−5))
        if i ≥ 10
            best := min(best, 1+COIN(i−10))
        if i ≥ 25
            best := min(best, 1+COIN(i−25))
        MC[i] := best
    return MC[n]
```

Runtime: $\Theta(\boldsymbol{n})$

Cashier’s algorithm

But there’s a simpler way! Just be **greedy**!

Cashier's algorithm

But there's a simpler way! Just be **greedy**!

```
CASHIER(n):
    while n > 0
        if n ≥ 25
            take quarter, n := n − 25
        else if n ≥ 10
            take dime, n := n − 10
        else if n ≥ 5
            take nickel, n := n − 5
        else
            take penny, n := n − 1
```

Cashier's algorithm

But there's a simpler way! Just be **greedy**!

$\textsc{Cashier}(n)$:

while $n > 0$

if $n \geq 25$

take quarter, $n := n - 25$

else if $n \geq 10$

take dime, $n := n - 10$

else if $n \geq 5$

take nickel, $n := n - 5$

else

take penny, $n := n - 1$

Why does it work???